

HYDRA: Explainability-Driven Intrusion Detection for IoT Network-Flow Data

Varatt Saengsiripongpun

Bachelor of Science (Honours) in
Computer Science and Artificial Intelligence

The University of Bath

2026

HYDRA: Explainability-Driven Intrusion Detection for IoT Network-Flow Data

Submitted by: Varatt Saengsiripongpun

Copyright

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see <https://www.bath.ac.uk/publications/university-ordinances/attachments/university-of-bath-ordinances.pdf>).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

Machine-learning-based Intrusion Detection Systems (IDS) achieve high benchmark performance yet frequently operate as opaque black boxes, issuing alerts without reliable justification. This opacity creates direct operational costs: analysts must investigate alerts without trustworthy guidance, false positives generate alert fatigue, and trust in automated detection degrades even when headline accuracy is strong.

This dissertation presents HYDRA, an explainability-driven IDS framework that treats model explanations as measurable system outputs. HYDRA addresses two tasks simultaneously: binary attack detection and multi-class attack-type classification, reflecting the operational need to know both that an anomaly has occurred and which threat category it represents. The core model set comprises majority-class and threshold baselines, Logistic Regression (as a reference explainer), Random Forest, XGBoost, and Gradient Boosted Trees, evaluated on TON_IoT as the primary benchmark and CIC-IoT2023 for generalisation. A CNN-LSTM model is designed as a deep learning extension; a Graph Neural Network is scoped as future work.

Explanations are generated using a portfolio of post-hoc methods matched to each model family: SHAP TreeExplainer for the tree ensembles, SHAP LinearExplainer for Logistic Regression, and LIME applied model-agnostically across all models. Integrated Gradients is specified for the CNN-LSTM extension. Every explanation method is evaluated against five measurable criteria: faithfulness (comprehensiveness and sufficiency under top- k feature ablation), stability (Spearman rank correlation under noise perturbation and across independent retraining runs), simplicity (k_{90} fraction and Gini coefficient), plausibility (RMA@ k against expert network features), and timeliness (milliseconds per sample). Identifier-based leakage from IP addresses and port numbers is controlled through two explicit feature regimes.

The contribution is not a novel detection algorithm but a reproducible evaluation framework that operationalises explainability as a measurable system property, enabling rigorous assessment of whether high-performing IDS models can provide explanations that are faithful, stable, and useful enough to support analyst decision-making in practice.

Contents

Abstract	2
Acknowledgements	1
1 Introduction	2
1.1 Motivation	2
1.2 Problem Statement	3
1.3 Aim and Objectives	3
1.4 Research Questions	4
1.5 Contributions	5
1.6 Dissertation Structure	5
2 Literature, Technology and Data Survey	6
2.1 Introduction	6
2.2 Evolution of Intrusion Detection: From Signatures to AIOps	6
2.2.1 Signature-Based and Rule-Based Systems	6
2.2.2 Anomaly-Based Detection and Machine Learning	7
2.2.3 AIOps for Cybersecurity	7
2.3 Supervised and Unsupervised Learning for Cyber Defence	8
2.3.1 Supervised Ensemble Methods	8
2.3.2 Isolation Forest	9
2.3.3 Autoencoders and LSTM-Autoencoders	9
2.4 Deep Learning for Log and Network-Flow Analysis	10
2.4.1 LSTM-Based Log Analysis: DeepLog	10
2.4.2 Transformer-Based Log Analysis: LogBERT and Variants	10
2.4.3 CNN-LSTM Ensembles for Network Flow Analysis	11
2.5 Graph-Based Detection and Predictive Analytics	11
2.5.1 Graph Neural Networks for Intrusion Detection	11
2.5.2 Predictive Analytics: Forecasting Cyber Threats	12
2.6 Explainability in Intrusion Detection	12
2.6.1 The Explainability Problem in IDS	12
2.6.2 SHAP TreeExplainer: Tree Ensemble Attribution	13
2.6.3 SHAP LinearExplainer: Logistic Regression Attribution	14

2.6.4	LIME: Model-Agnostic Cross-Reference	14
2.6.5	Integrated Gradients: CNN-LSTM Attribution	15
2.6.6	GNNExplainer: Graph Neural Network Attribution	15
2.6.7	Policy Graphs and Causal Graph Explainability	16
2.6.8	XAI Evaluation: Five Criteria	16
2.7	Positioning HYDRA Against Concurrent Work	18
2.8	Gaps in Current Technology	20
2.8.1	Evaluation Leakage and Overoptimistic Benchmarks	20
2.8.2	Alert Fatigue and False Positive Management	21
2.8.3	Binary-Only Detection and Limited Attack Specificity	21
2.8.4	Explainability as a Qualitative Afterthought	21
2.8.5	Adversarial Robustness and Concept Drift	21
2.9	Technology Survey	22
2.9.1	Streaming Infrastructure: Apache Kafka	22
2.9.2	Machine Learning Frameworks	22
2.9.3	Explainability Libraries	22
2.9.4	Visualisation and Dashboard Tools	23
2.10	Data Survey	23
2.10.1	Overview	23
2.10.2	TON_IoT (Primary Dataset)	23
2.10.3	CIC-IoT2023 (Secondary Generalisation Dataset)	24
2.10.4	LogHub (Supporting Reference)	24
2.10.5	LANL Unified Host and Network Dataset (Contextual Reference)	25
2.10.6	Data Preparation Principles	25
2.11	Summary	25
3	Discussion	27
3.1	Recap of Key Results	27
3.2	Interpretation of Results	27
3.3	Practical Implications	27
3.4	Reflection on Research Questions	27
3.5	Limitations	27
4	Conclusions	29
4.1	Summary of Contributions	29
4.2	Achievement of Aim	29
4.3	Future Work	30
4.4	Closing Remarks	30
A	Feature Groups for TON_IoT	37

B	Explainability Metric Formulae	38
C	Acknowledgement of Generative AI Use	41

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Julian Padget, for his expert guidance, detailed feedback, and sustained encouragement throughout this project. His interdisciplinary perspective across artificial intelligence, formal methods, and security systems has been invaluable in shaping the scope and rigour of this work.

I am grateful to the Department of Computer Science at the University of Bath for the academic environment and computational resources that supported this research.

I also thank the research communities behind the datasets used in this project—particularly the UNSW Canberra Cyber Security Group for the TON_IoT dataset and the Canadian Institute for Cybersecurity for CIC-IoT2023—whose publicly available resources form the empirical foundation of this dissertation.

Finally, I acknowledge the use of generative AI tools for text editing and structural suggestions during the drafting process, in accordance with the University of Bath's Type B guidelines on AI-assisted assessment. All technical ideas, design decisions, experimental methodology, analysis, and written conclusions are the author's own work.

Chapter 1

Introduction

1.1 Motivation

The cyber-defence landscape of 2025 is defined by a fundamental asymmetry: defenders must secure an ever-expanding and heterogeneous attack surface, while adversaries require only a single successful compromise. This imbalance is sharpening rapidly. Industrial-scale automated scanning and exploitation infrastructure now operates at tens of thousands of scans per second, and credential-based attacks have risen markedly in recent years [22]. Simultaneously, enterprises generate billions of log and telemetry events per day across servers, endpoints, cloud services, and IoT devices, creating a signal-to-noise problem that overwhelms both traditional rule-based monitoring and human analysts working at scale [32].

Conventional security controls—signature-based Intrusion Detection Systems (IDS), rule-driven Security Information and Event Management (SIEM) platforms, and manual incident triage—were never designed for this operating environment. Signature-based systems such as Snort and Suricata offer high precision for known threats but are fundamentally reactive: they cannot detect previously unseen attacks, polymorphic malware, or adversaries who operate using legitimate credentials and living-off-the-land techniques [13]. SIEM platforms aggregate and correlate events but depend on static rule sets whose maintenance overhead grows with system complexity, and which produce high false positive rates when alert thresholds are set conservatively [61].

The emerging discipline of AIOps for cybersecurity seeks to address these limitations by embedding machine learning, automation, and streaming analytics into the Security Operations Centre (SOC). Industry analyses suggest that mature AIOps platforms can reduce alert noise by 70–90% through automated correlation and deduplication, and can significantly improve mean time to detect (MTTD) and mean time to respond (MTTR) to incidents [11, 32]. However, the adoption of learned models introduces new operational challenges: deep learning systems frequently

behave as opaque black boxes; real-time inference at SOC scale imposes strict latency constraints; and models are vulnerable to concept drift and adversarial manipulation in hostile environments [27, 59].

1.2 Problem Statement

A central failing of current IDS research is the gap between detection performance and operational usefulness. High benchmark accuracy on public datasets does not translate directly into deployable, trustworthy systems for three interconnected reasons.

First, many published results are inflated by evaluation leakage. When network-flow records are split randomly by row, highly similar traffic from the same hosts, sessions, or adjacent time windows appears in both training and test sets, allowing models to achieve near-perfect scores by recognising identity patterns—specific IP addresses, port combinations, or session fingerprints—rather than learning attack behaviour [57]. This type of leakage is invisible under standard evaluation but would cause catastrophic performance degradation in deployment against unseen infrastructure.

Second, most IDS models provide binary outputs—attack or normal—without indicating which category of threat has been detected. In practice, different attack types such as denial-of-service, ransomware, data exfiltration, and scanning require different analyst responses and escalation paths. A system capable of both binary detection and multi-class attack-type classification is therefore substantially more operationally useful.

Third, even accurate models are frequently dismissed or underutilised because their alerts lack justification. When a model flags a flow as malicious without explaining which behavioural features triggered the decision, analysts must investigate blindly. This increases mean time to triage, compounds alert fatigue, and erodes the trust required for automation to be accepted in high-stakes SOC environments.

1.3 Aim and Objectives

The aim of this project is to design, implement, and evaluate HYDRA: an intrusion detection framework that addresses both binary attack detection and multi-class attack-type classification, while generating per-alert explanations that are systematically validated for reliability, consistency, and operational usefulness on IoT network-flow data.

The specific objectives are:

1. To implement and evaluate a structured model progression—from baselines through tabular ensemble models—for both binary and multi-class intrusion detection on TON_IoT, with CIC-IoT2023 as a secondary generalisation benchmark. A CNN-LSTM model is additionally designed and specified as a deep learning extension; a Graph Neural Network is scoped as future work.
2. To define and apply two feature regimes that explicitly isolate and quantify the contribution of identifier leakage to detection performance and explanation quality.
3. To generate standardised per-alert explanation records for tabular models using SHAP TreeExplainer, for the Logistic Regression reference model using SHAP LinearExplainer, and for all models using LIME as a model-agnostic cross-reference. Integrated Gradients is specified for the CNN-LSTM extension.
4. To evaluate explanations systematically using five measurable criteria: faithfulness (comprehensiveness and sufficiency), stability (under noise perturbation and across independent retraining seeds), simplicity (k_{90} and Gini), plausibility ($\text{RMA}@k$), and timeliness (ms per sample).
5. To position HYDRA explicitly against concurrent XAI-IDS work and compare the performance–explainability trade-off across model types, with generalisation evaluation on CIC-IoT2023 where feasible.

1.4 Research Questions

This project is structured around four research questions:

RQ1 (Detection): Which modelling approach provides the strongest performance for both binary attack detection and multi-class attack-type classification on tabular network-flow features, when evaluated using operationally relevant metrics such as PR-AUC and false positive rate at a fixed recall target?

RQ2 (Explainability): To what extent can the best-performing detector provide per-alert, feature-level explanations with high and consistent coverage across both detection tasks?

RQ3 (Reliability): How stable are explanation outputs across repeated training runs and different data conditions, and does stability differ between the binary and multi-class tasks?

RQ4 (Faithfulness): Do model-generated explanations exhibit fidelity—that is, does removing the highlighted features significantly change model confidence—and do

explanation patterns align with known network intrusion behaviours for each attack type?

1.5 Contributions

This project does not propose a novel detection algorithm. Its contributions are:

- A reproducible IDS pipeline that supports both binary and multi-class intrusion detection with standardised per-alert explanation records for tabular ensembles, with a CNN-LSTM extension designed for future evaluation.
- A unified, implementation-ready explainability evaluation protocol covering five measurable criteria (faithfulness, stability, simplicity, plausibility, timeliness), applied consistently across all evaluated model families and both detection tasks. Importantly, the stability criterion is operationalised as two separate experiments: seed-retraining stability (training $S = 5$ models with different random seeds) and noise-perturbation robustness (perturbing fixed model inputs), which address distinct aspects of explanation reliability.
- Empirical analysis of the impact of identifier-based feature leakage on detection performance and explanation quality in IoT network-flow datasets, and explicit positioning of HYDRA against concurrent XAI-IDS systems through a gap analysis table.

1.6 Dissertation Structure

Chapter 2 presents the literature, technology, and data survey, covering the evolution of intrusion detection and AIOps, supervised and unsupervised learning for cyber defence, deep learning approaches to log and flow analysis, predictive analytics, explainability methods, and a critical survey of the datasets used. Chapter 3 describes the HYDRA methodology. Chapter 4 presents experimental results. Chapter 5 discusses findings, trade-offs, and limitations. Chapter 6 concludes and identifies future work. A full bibliography and appendices follow.

Chapter 2

Literature, Technology and Data Survey

2.1 Introduction

This chapter reviews the literature, technology landscape, and available datasets relevant to HYDRA. Section 2.2 traces the evolution of intrusion detection from signature-based systems through machine-learning anomaly detection to AIOps. Section 2.3 examines supervised and unsupervised learning algorithms used in cyber defence. Section 2.4 covers deep learning approaches to log and network-flow analysis. Section 2.5 reviews predictive analytics for cyber threats. Section 2.6 surveys explainability methods, with particular focus on post-hoc feature attribution and graph-based explanation. Section 2.8 identifies critical gaps in existing technology that motivate HYDRA’s design. Section 2.9 surveys key technologies used in the implementation. Section 2.10 surveys the datasets used in evaluation.

2.2 Evolution of Intrusion Detection: From Signatures to AIOps

2.2.1 Signature-Based and Rule-Based Systems

Early intrusion detection technology was dominated by signature-based systems such as Snort and Suricata, which compare packet headers and payloads against libraries of known Indicators of Compromise (IoCs). These systems offer high precision for threats whose signatures are already catalogued and low computational overhead. However, their fundamental limitation is their inability to detect previously unseen attacks. Signature databases must be manually maintained, and any attack variant that differs from the known pattern—through obfuscation, encryption, or minor modification—will evade detection entirely. As adversaries increasingly rely on encrypted channels, legitimate system tools, and credential theft rather than novel exploits, the coverage of static signatures continues to erode [13].

Rule-based SIEM platforms extend signature detection by correlating events across sources and applying logic rules to identify multi-step attack sequences. However, Zhu et al. [61] highlight that as software complexity increases, static parsing rules become brittle: they require continuous manual update, generate high false positive rates when applied to heterogeneous modern telemetry, and miss anomalies that do not match any predefined pattern. The maintenance burden of rule sets scales poorly with the diversity and volume of data produced by modern enterprise and IoT environments.

2.2.2 Anomaly-Based Detection and Machine Learning

To address the zero-day detection gap, research shifted towards anomaly-based IDS, where models learn a statistical baseline of normal behaviour and flag deviations. Early statistical methods such as threshold-based detection and principal component analysis provided a foundation, but their performance was limited on high-dimensional, non-linear network traffic data.

The application of machine learning to IDS marked a significant advance. Supervised approaches such as Random Forest, Support Vector Machines, k -Nearest Neighbours, and gradient boosting methods demonstrated high accuracy on benchmark datasets such as KDD Cup 99, NSL-KDD, and CICIDS2017 when applied to engineered flow features [14]. These models benefit from interpretable feature importance scores and mature implementations, making them competitive baselines that remain difficult to surpass in tabular settings even with more complex architectures. However, supervised models generalise poorly to attack types not represented in training data and suffer performance degradation under concept drift as network behaviour evolves.

Hybrid IDS architectures that combine supervised classification with unsupervised anomaly detection have emerged as a research direction with particular promise. Alsaedi et al. [2] show that combining tree-based supervised models with autoencoder-based unsupervised detection on the TON_IoT dataset yields substantially better recall on zero-day scenarios than either approach alone, while also reporting improvements in both precision and detection coverage. The hybrid modelling principle underpins HYDRA's experimental design.

2.2.3 AIOps for Cybersecurity

AIOps (Artificial Intelligence for IT Operations) emerged as a broader discipline for automating IT operations through machine learning, focusing on noise reduction, automated correlation, and root-cause analysis across heterogeneous telemetry [56].

For cybersecurity, AIOps platforms ingest security signals from SIEMs, IDS, endpoint detection and response (EDR) systems, and cloud logging pipelines into unified analytical frameworks that operate continuously on streaming data [11]. Industry reports suggest that mature AIOps deployments can reduce alert noise by 70–90% through automated deduplication and correlation, and can improve mean time to detect and mean time to respond to incidents significantly [32, 56]; while such figures originate from vendor analyses rather than peer-reviewed studies, the directional benefit of automated correlation is broadly supported in the research literature [56, 61].

Commercial AIOps and SIEM platforms from vendors including Datadog, Splunk, IBM QRadar, and CrowdStrike incorporate ML-based anomaly detection, user and entity behaviour analytics (UEBA), and increasingly, graph-based attack-path visualisation. However, these products are closed, vendor-specific, and optimised for enterprise operational metrics rather than research-grade reproducibility or publication of detailed detection and prediction benchmarks on standardised open datasets. HYDRA’s contribution is a transparent, research-reproducible prototype that addresses the same operational goals while enabling rigorous empirical evaluation.

2.3 Supervised and Unsupervised Learning for Cyber Defence

2.3.1 Supervised Ensemble Methods

Ensemble tree methods form the core of HYDRA’s tabular detection pipeline. Random Forest [8] constructs an ensemble of independently trained decision trees by combining bootstrap sampling with random feature subsets at each split, producing stable feature importance estimates well-suited to high-dimensional flow data. XGBoost [10] implements gradient boosted trees through second-order gradient approximation and regularisation, offering strong generalisation and efficient computation. LightGBM [33] extends this through histogram-based splitting for memory efficiency. Logistic Regression is also included in HYDRA’s model set as a reference explainer: its linear decision boundary makes it deterministic and auditable by construction, providing a stability anchor against which the explanation reliability of the more complex ensemble models can be measured.

In the IDS literature, ensemble models consistently achieve state-of-the-art results on public benchmarks. One study [28] integrates Restricted Boltzmann Machines, Deep Feedforward Neural Networks, LSTMs, and GRUs alongside Random Forest and XGBoost, reporting 99.9% accuracy on CICDDoS2019 and HIKARI2021. A separate

study [29] demonstrates that integrating Random Forest with deep artificial neural network components yields strong intrusion detection performance across multiple attack categories, outperforming standalone tree or neural network baselines. While these results appear impressive, both use datasets without explicit temporal splits, raising concerns about evaluation leakage that limit the generalisability of reported metrics.

A key challenge for supervised IDS is class imbalance. In realistic deployments, attack events represent a small fraction of total traffic. Standard accuracy metrics are therefore misleading: a classifier that always predicts *normal* achieves very high accuracy while detecting nothing. Operational evaluation must rely on precision-recall metrics, PR-AUC, and false positive rate at a fixed recall target, which directly reflect the burden on analysts who must investigate every raised alert.

2.3.2 Isolation Forest

Isolation Forest [35] is a widely-used unsupervised anomaly detection algorithm well-suited to high-dimensional tabular data. Rather than profiling normal instances, it isolates anomalies directly by recursively partitioning the feature space with random cuts: anomalous points, which occupy sparse or extreme regions, require fewer partitions to isolate and receive shorter average path lengths across an ensemble of random trees. The algorithm operates in approximately linear time and memory, making it practical as a fast first-stage filter in a hybrid pipeline.

Alsaedi et al. [2] demonstrate Isolation Forest’s utility as a pre-filter on the TON_IoT dataset, showing that combining it with downstream supervised classification reduces false positives while maintaining recall comparable to supervised-only approaches. A comparative study [58] confirms that Isolation Forest outperforms autoencoders for detecting point anomalies in tabular data, while autoencoders are more effective for contextual anomalies in sequential or high-dimensional embedding spaces.

2.3.3 Autoencoders and LSTM-Autoencoders

Autoencoders are neural networks trained to compress input data into a low-dimensional latent representation and reconstruct it. When trained exclusively on normal traffic, an autoencoder learns to reconstruct normal patterns efficiently; anomalous inputs produce high reconstruction error, which serves as the anomaly score. Deep autoencoders are particularly effective for detecting subtle deviations in high-dimensional IoT telemetry data where simple threshold-based methods fail [42].

LSTM-autoencoders extend this approach to sequential data by using Long Short-

Term Memory networks as both encoder and decoder, allowing the model to capture temporal dependencies in log sequences or time-windowed traffic metrics. Studies on time-series anomaly detection demonstrate that LSTM-autoencoders can detect subtle temporal anomalies that evade point-level detection [38]. Park et al. [45] demonstrate that LSTM-autoencoders outperform classic anomaly detection baselines such as Local Outlier Factor on time-series network flow data, at the cost of higher training complexity and sensitivity to hyperparameter choices.

2.4 Deep Learning for Log and Network-Flow Analysis

2.4.1 LSTM-Based Log Analysis: DeepLog

System logs represent structured or semi-structured text describing system state and events. The application of Natural Language Processing (NLP) techniques to log analysis has produced a series of increasingly capable models. DeepLog [15] treats log entries as sequences of discrete log keys—templates derived by parsing variable-length log messages into fixed event identifiers—and trains an LSTM to predict the next expected key in a sequence. Deviations from the predicted key, or sequences with low predicted probability, are flagged as anomalies.

DeepLog demonstrated strong performance on the HDFS and BGL datasets from the LogHub collection [61], establishing LSTMs as a viable architecture for log-based anomaly detection. However, LSTMs are susceptible to two structural limitations: they struggle to model very long-range dependencies without careful regularisation, and they are trained unidirectionally, meaning they can only condition predictions on the left context of a sequence. As a result, DeepLog may miss anomalies that are only detectable from subsequent context.

2.4.2 Transformer-Based Log Analysis: LogBERT and Variants

More recent work applies Transformer architectures to log anomaly detection. LogBERT [25] adapts BERT to the log domain by treating log key sequences as sentences and applying masked log key prediction as a self-supervised pre-training objective. Unlike DeepLog, LogBERT conditions each prediction on both left and right context, enabling it to detect anomalies that depend on future events. Guo et al. [25] report superior F1-scores on HDFS and BGL compared to LSTM baselines, with particular improvements in detecting anomalies involving unusual key co-occurrences.

Comparative systematic evaluations confirm that Transformer-based models achieve higher accuracy than LSTM-based approaches on LogHub benchmarks, but at substantially higher computational cost for both training and inference [25, 54]. This

latency–accuracy trade-off is directly relevant to real-time IDS deployment: in scenarios requiring very low detection latency, LSTM-based models or lightweight alternatives such as LogFiT [37] and ALogSCAN [1] may be preferable.

2.4.3 CNN-LSTM Ensembles for Network Flow Analysis

For network flow data, which has both spatial (feature interaction) and temporal (sequence) structure, hybrid CNN-LSTM architectures have been proposed to capture both dimensions simultaneously. Convolutional layers extract local feature patterns from flow-level attributes, while LSTM layers model temporal dependencies across successive time windows or flow sequences. Studies using the TON_IoT dataset demonstrate that LSTM-based approaches—including CNN-LSTM ensembles and bidirectional LSTM variants—achieve detection accuracy above 99% on binary classification, with strong performance maintained across multiple IoT attack types including DoS, DDoS, ransomware, and data exfiltration [30, 41].

A more recent architecture [26] combines three parallel branches—CNN for spatial patterns, BiLSTM for temporal dependencies, and an autoencoder for reconstruction-error anomaly scoring—fused into a single detection model trained using federated learning. The federated approach keeps raw IoT traffic data local to each device, sharing only model updates, which addresses privacy constraints in distributed IoT environments.

2.5 Graph-Based Detection and Predictive Analytics

2.5.1 Graph Neural Networks for Intrusion Detection

Network communications have natural graph structure: hosts and devices are nodes, and communications between them are edges. Graph Neural Networks (GNNs) can exploit this relational structure by aggregating information from neighbouring nodes through successive message-passing operations, learning representations that capture not just individual flow features but the topology of communication patterns. This is particularly relevant for detecting attack behaviours that manifest at the graph level, such as scanning (a single source node connecting to many previously unseen destinations), lateral movement (a host establishing unusual connections to internal targets), and botnet coordination (many nodes exhibiting correlated behavioural patterns).

On the LANL dataset, temporal GNN models such as Euler [34] and EarlyDLM [36] have been applied to user authentication graphs, where nodes represent users and machines and edges represent authentication events over time. These models predict

the probability of each user-machine authentication being anomalous given the historical communication topology, achieving strong performance on insider threat and lateral movement detection scenarios.

2.5.2 Predictive Analytics: Forecasting Cyber Threats

Beyond detection of ongoing attacks, there is substantial interest in predicting imminent threats before they fully materialise. Munkhdorj et al. [43] propose using social data and vulnerability intelligence to estimate the probability of an impending cyber attack. Recurrent sequence models have also been applied to volumetric threat forecasting: LSTM-based approaches have been shown to forecast DDoS attack intensity up to several minutes in advance using historical traffic time series [4].

Lateral movement prediction is a closely related problem with particular relevance to enterprise intrusion scenarios. Using the LANL dataset, Turcotte, Kent and Hash [57] train sequence models on historical user authentication patterns to predict the next machine a user is likely to authenticate to. When a user authenticates to a machine with low predicted probability, this signals potential credential theft or lateral movement by an attacker using stolen credentials. This technique is notable because it can detect attacks that produce no direct signature—the attacker uses valid credentials and legitimate system tools—and is thus invisible to signature-based and many statistical anomaly detection approaches.

A survey of event prediction methods for cybersecurity [53] identifies three primary metrics for evaluating predictive IDS: lead time (the interval between prediction and incident), time-to-compromise estimation, and prediction reliability under class imbalance and concept drift.

2.6 Explainability in Intrusion Detection

2.6.1 The Explainability Problem in IDS

High-performing ML models used for intrusion detection—particularly gradient boosted trees and deep learning architectures—frequently operate as black boxes, producing alert decisions without exposing the features or reasoning that drove them. This is problematic for several operational reasons. Analysts must investigate each alert without guidance, increasing triage time. False positives—which remain a persistent challenge even for well-performing models—are harder to identify and dismiss without understanding why the model flagged a given flow. When models are retrained or updated, analysts have no basis for assessing whether the new model has learned different or better decision rules. And in regulated sectors, security

decisions may require audit trails that document the reasoning behind automated actions.

The field of Explainable AI (XAI) has developed two broad paradigms for addressing this problem. Post-hoc methods explain an already-trained model by approximating its behaviour locally or globally, without modifying the model itself. Intrinsically interpretable models are designed to be transparent by construction, at the potential cost of detection performance. For IDS, post-hoc methods are more widely used because they can be applied to high-performing ensemble and deep learning models, whereas intrinsically interpretable models such as logistic regression may not capture the non-linear patterns needed for strong detection [17].

HYDRA implements a portfolio of four complementary post-hoc explanation methods across its model set: SHAP TreeExplainer for tree-based ensemble models (Random Forest, XGBoost, Gradient Boosted Trees), SHAP LinearExplainer for the Logistic Regression reference model, LIME applied across all models as a model-agnostic cross-reference, and Integrated Gradients for the CNN-LSTM and GNNExplainer for the Graph Neural Network. This design allows direct comparison of explanation quality across model families and across methods applied to the same model.

2.6.2 SHAP TreeExplainer: Tree Ensemble Attribution

SHAP (SHapley Additive exPlanations), introduced by Lundberg and Lee [39], grounds feature attribution in cooperative game theory: each feature is treated as a player in a coalition game, and its contribution to the prediction is its Shapley value—the average marginal contribution it makes across all possible orderings of features. This provides theoretically grounded attribution that satisfies efficiency, symmetry, and dummy axioms, making it the most principled and widely adopted explanation method in the IDS literature [51].

SHAP TreeExplainer provides an exact, polynomial-time implementation specifically for tree-based models including Random Forest, XGBoost, and gradient boosted trees [40]. By exploiting the tree structure directly, TreeExplainer avoids the sampling approximation required by model-agnostic SHAP variants, making it both faster and more accurate. In HYDRA, TreeExplainer is applied to all three ensemble models and produces two types of output: local explanations that give per-alert feature contributions, and global explanations that aggregate Shapley values across the test set to produce feature importance rankings and per-attack-type heatmaps showing which features are most characteristic of each threat category.

Despite its strengths, SHAP has known limitations. Shapley values can be unstable under correlated features: when two features carry redundant information, their in-

dividual values may vary substantially across retraining even if the model’s decision boundary is stable. This is addressed in HYDRA’s evaluation through explicit stability measurement using Spearman rank correlation of feature importance orderings under noise perturbation.

2.6.3 SHAP LinearExplainer: Logistic Regression Attribution

For the Logistic Regression model, SHAP LinearExplainer computes Shapley values analytically for linear models by exploiting the additive structure of linear prediction: the attribution of each feature is its coefficient multiplied by the deviation of its value from the expected value. Because Logistic Regression is a linear model, its explanations are deterministic across training runs given the same data and solver settings, making it an ideal stability anchor for comparing against the more complex tree ensemble models. The primary purpose of Logistic Regression in HYDRA is therefore as a reference explainer rather than as a competitive detector. The deviation of RF, XGBoost, and GBT explanations from this anchor quantifies the explainability cost of moving to more expressive models, consistent with the performance–explainability trade-off analysis of Bhatt et al. [7] and the explainable ensemble survey [17].

2.6.4 LIME: Model-Agnostic Cross-Reference

LIME (Local Interpretable Model-agnostic Explanations), introduced by Ribeiro, Singh and Guestrin [50], explains individual predictions by sampling perturbed versions of the input, querying the model on each perturbation, and fitting a locally linear surrogate model to the resulting input-output pairs. LIME is model-agnostic and can therefore be applied identically across all of HYDRA’s models, enabling direct comparison of local explanations across model families on the same alert.

HYDRA applies LIME across all models for two reasons. First, it provides a model-agnostic cross-reference against TreeSHAP and LinearSHAP attributions: significant disagreements between LIME and SHAP for the same alert indicate explanation instability that may reflect either method’s limitations rather than genuine model reasoning. Second, LIME’s application to the CNN-LSTM provides tabular-level local explanations that complement the gradient-based Integrated Gradients attributions applied to the same model.

LIME’s primary limitation is sampling variance: the locally linear surrogate is fitted to a finite random sample of perturbations, and different samples may yield qualitatively different explanations for the same instance. Comparative studies in IDS contexts generally find SHAP to provide more stable attributions for tabular data [65]. HYDRA therefore treats LIME as a secondary cross-reference method rather than a

primary attribution source.

2.6.5 Integrated Gradients: CNN-LSTM Attribution

Integrated Gradients [52] is a gradient-based attribution method for differentiable models. It computes the attribution of each input feature by integrating the model’s gradients with respect to that feature along a straight-line path from a baseline input (typically a zero vector or mean-value reference) to the actual input. This integration ensures that the method satisfies sensitivity (a feature that changes the output must receive non-zero attribution) and implementation invariance (functionally equivalent models receive the same attributions regardless of implementation).

In HYDRA, Integrated Gradients is applied to the CNN-LSTM model, which combines convolutional layers for local feature pattern extraction with LSTM layers for temporal dependency modelling across time-windowed flow sequences. The CNN-LSTM and its associated explanation methods are planned as a later-stage component of HYDRA and have not yet been fully evaluated at the time of writing.

2.6.6 GNNExplainer: Graph Neural Network Attribution

Explaining GNN predictions requires methods that account for both graph topology (which nodes and edges drove the prediction) and feature content (which attributes of those nodes and edges were decisive). GNNExplainer [60] addresses the topology problem by learning a soft mask over the edges of a node’s computational subgraph that maximises the mutual information between the masked subgraph’s prediction and the full-graph prediction, subject to a sparsity constraint. The result is a compact explanation subgraph that approximately preserves the model’s prediction for the instance being explained.

GNN explanations face unique challenges not present in tabular settings. Subgraph explanations can be non-unique: multiple subgraphs may achieve equivalent mutual information with the prediction. Explanations may be dominated by graph topology rather than edge-feature content, making them uninformative about which behavioural attributes drove the detection. And unlike TreeSHAP, GNNExplainer is an optimisation-based method that introduces additional stochasticity, potentially producing different explanations across runs even for the same model and input. HYDRA addresses this by explicitly measuring subgraph explanation stability using the same Spearman rank correlation protocol applied to other models. Like the CNN-LSTM, the GNN and its explanation pipeline are planned as a later-stage component.

2.6.7 Policy Graphs and Causal Graph Explainability

Beyond feature attribution, a more structured approach to IDS explainability uses graph representations of security policies and causal event chains. Policy graphs [6] represent access control states as directed graphs of entities and relationships, explaining alerts by identifying which entity-relationship pattern triggered a policy violation. Causal or provenance graphs represent the history of system events as dependency graphs. DepImpact [19] proposes backward-propagation over system dependency graphs to isolate the minimal subgraph most relevant to an observed attack. UTLParser [55] combines log parsing with causal graph construction for attack attribution.

HYDRA’s primary explainability layer uses the post-hoc statistical methods described above, but the policy and provenance graph paradigm directly informs the plausibility evaluation criterion: explanation features that align with known policy violations or causal attack chains are considered operationally plausible, providing a domain-knowledge reference for the $RMA@k$ metric.

2.6.8 XAI Evaluation: Five Criteria

A recurring weakness in XAI research applied to IDS is the absence of systematic evaluation of explanations. Most studies present SHAP plots or attention maps as qualitative demonstrations, without measuring whether explanations are reliable, faithful, or operationally useful. HYDRA addresses this gap by evaluating every explanation method applied to every model against five criteria, each with a specific metric, as summarised in Table 2.1.

Faithfulness is measured through top- k feature ablation using two complementary tests. Comprehensiveness removes the top- k attributed features from the input and measures the resulting drop in model confidence. Sufficiency retains only the top- k features and replaces all others with baseline values, measuring whether the retained features alone are sufficient to approximately reproduce the original confidence. Together, these provide a stronger faithfulness test than either alone [7, 12].

Stability uses Spearman rank correlation rather than Jaccard set overlap because it captures the full ordering of feature importance—a more sensitive measure of explanation consistency—and is robust to ties. Perturbations are applied as small Gaussian noise added to continuous features, simulating the measurement noise present in real network flow monitoring. Alvarez-Melis and Jaakkola [3] demonstrate that many local explanation methods are sensitive to small changes in model parameters, producing qualitatively different explanations even when the decision boundary is largely unchanged. Viirlaid et al. [65] confirm this instability empirically

Table 2.1: HYDRA’s five-criterion XAI evaluation framework, applied to each explanation method across each model.

Criterion	Metric	Description
Faithfulness	Comprehensiveness + Sufficiency (top- k feature ablation)	Remove top- k features (comprehensiveness) or keep only top- k (sufficiency) and measure confidence change. High-quality explanations cause large drops when top- k features are removed and small drops when only they are retained.
Stability	$S1$: Spearman rank correlation across $S = 5$ independent re-training runs; $S2$: Spearman correlation under Gaussian input noise	Two distinct experiments. $S1$ measures robustness to training stochasticity (addresses RQ3). $S2$ measures robustness to input perturbation (addresses RQ4 robustness). Both are reported separately.
Simplicity	k_{90} fraction + Gini coefficient	k_{90} is the minimum number of features needed to account for 90% of total attribution mass, normalised by feature count. Gini coefficient measures concentration of attribution. Lower k_{90} and higher Gini indicate more concise, analyst-friendly explanations.
Plausibility	RMA@ k vs expert network features	Relevant-feature Match Accuracy at k : the proportion of top- k attributed features that appear in an expert-defined reference set known to be characteristic of each attack type.
Timeliness	Milliseconds per sample	Wall-clock time to generate a complete explanation record for a single alert. Constrains which methods are viable for real-time or near-real-time SOC deployment.

in an NIDS context, finding that SHAP and LIME attributions vary meaningfully across retraining runs, reinforcing the need for HYDRA’s explicit seed-retraining stability protocol.

Simplicity is quantified using two complementary measures. The k_{90} fraction is the minimum number of top-attributed features needed to account for 90% of the total attribution mass, normalised by the total feature count. The Gini coefficient of the attribution distribution measures concentration: a high Gini value indicates that attribution mass is concentrated on a small number of features, while a low value indicates diffuse attribution that is harder to interpret.

Plausibility is evaluated using Relevant-feature Match Accuracy at k (RMA@ k), computed by comparing the top- k attributed features for each alert against a reference set of features that network security domain knowledge identifies as characteristic of the detected attack type. The reference sets are defined in advance from the IDS literature and used as a fixed evaluation benchmark, not tuned against model outputs.

Timeliness measures the wall-clock time required to generate a complete explanation record for a single alert at inference time, reported in milliseconds per sample. TreeSHAP is generally very fast for tree models; LIME, which requires repeated model queries for each explained sample, is substantially slower; and GNNExplainer, which solves an optimisation problem per explanation, may be the slowest. Timeliness measurement allows HYDRA to characterise these trade-offs empirically.

This five-criterion framework is applied consistently across all explanation method-model pairs in HYDRA, enabling structured comparison of explanation quality between tabular and graph models, between primary and cross-reference methods, and between the binary and multi-class detection tasks.

2.7 Positioning HYDRA Against Concurrent Work

Four recent papers address XAI-IDS in settings sufficiently similar to HYDRA that explicit positioning is necessary. Table 2.2 maps each paper against HYDRA’s principal design choices across ten dimensions. The table makes clear that while individual components of HYDRA’s approach appear in the literature, no prior work combines all of them simultaneously: dual-task detection (binary and multi-class), leakage-controlled evaluation, a five-criterion measurable XAI framework, a multi-method explanation portfolio, and cross-dataset generalisation.

E-XAI (2024) [62] is the closest concurrent work. It evaluates SHAP, LIME, and Integrated Gradients on a deep-learning-based NIDS on UNSW-NB15 and CICIDS2017,

Table 2.2: Positioning HYDRA against concurrent XAI-IDS work. ✓ = present, ◦ = partial, × = absent.

	HYDRA	E-XAI [62]	XAI-IDS [63]	Frontiers [64]	TalTech [65]
Binary + multi-class detection	✓	×	×	×	×
Leakage-controlled feature regimes	✓	×	×	×	×
Temporal (non-random) splitting	✓	×	×	◦	×
≥3 XAI methods compared	✓	✓	✓	◦	✓
Stability measured quantitatively	✓	×	×	×	◦
Timeliness (ms/sample) measured	✓	×	×	×	×
Simplicity (k_{90} , Gini) measured	✓	×	×	×	×
Plausibility ($\text{RMA}@k$) measured	✓	×	×	×	×
IoT-specific dataset (TON_IoT)	✓	×	◦	✓	✓
Cross-dataset generalisation	✓	×	×	×	×

producing both local and global explanation outputs. However, E-XAI is framed as a binary classification task only and does not address multi-class attack-type identification, which HYDRA treats as a co-equal detection objective. E-XAI does not separate behaviour-only from identifier-inclusive features and reports no quantitative stability or timeliness measurements. Additionally, E-XAI evaluates models using random row-level splits without temporal ordering, a design choice that risks the identity-pattern leakage described in Section 2.8. The datasets used by E-XAI (UNSW-NB15 and CICIDS2017) are general enterprise network benchmarks, not IoT-specific; IoT traffic is characterised by fundamentally different flow patterns—short-lived connections, protocol-constrained communication, and device-specific behaviour—that make direct comparison with HYDRA’s TON_IoT evaluation inappropriate. Finally, E-XAI does not generalise across datasets, whereas HYDRA evaluates on a secondary benchmark (CIC-IoT2023) to test explanation portability. HYDRA’s dual-task design, leakage-controlled evaluation, five-criterion framework, and cross-dataset generalisation collectively represent a direct methodological advance beyond E-XAI.

XAI-IDS, Applied Sciences (2024) [63] presents an explainable AI framework for intrusion detection using both global and local SHAP and LIME explanations, evaluated on the CICIDS2017 dataset. The paper focuses on binary classification and applies random data splitting without temporal ordering. It does not address IoT-specific datasets, multi-class detection, leakage-controlled feature regimes, or any quantitative stability or timeliness measurement. HYDRA extends this work by making the XAI evaluation framework systematic and measurable across five defined criteria.

Frontiers in Computer Science (2025) [64] applies SHAP and LIME to an ML-based IDS on IoT networks. The evaluation is primarily qualitative and does not employ the five-criterion protocol or address leakage. HYDRA provides a more rigorous and

reproducible evaluation methodology for the same problem domain.

TalTech SOC XAI Study (2025) [65] is particularly relevant to HYDRA’s RQ4. It compares LIME, SHAP, Integrated Gradients, and DeepLIFT on NIDS alerts using a four-criterion framework (faithfulness, complexity, robustness, reliability) and finds DeepLIFT outperforming SHAP and LIME on faithfulness. This is a directly comparable finding: HYDRA does not implement DeepLIFT (which is specific to backpropagation-based deep models and is not applicable to tree ensembles), but the TalTech result provides an important benchmark for the CNN-LSTM extension, where Integrated Gradients and DeepLIFT address the same attribution problem. HYDRA’s addition of timeliness and simplicity to the evaluation criteria, and its IoT-specific focus, are key differentiators.

Two further concurrent works warrant acknowledgement. UXIEF [UXIEF_REF] proposes a conceptual trilemma framework characterising the tension between detection performance, resource efficiency, and explanation quality in IDS design; however, it provides no empirical implementation or experimental validation, remaining a theoretical contribution. ERI [ERI_REF] introduces a domain-agnostic explanation reliability index grounded in four stability axioms, offering a principled formalisation of explanation consistency; however, it does not address IDS-specific evaluation contexts, dual-task detection, or leakage-controlled feature regimes. Neither work therefore occupies the same empirical space as HYDRA. HYDRA is the first system to empirically implement and validate all ten dimensions simultaneously.

2.8 Gaps in Current Technology

2.8.1 Evaluation Leakage and Overoptimistic Benchmarks

A pervasive problem in published IDS research is evaluation leakage that artificially inflates reported performance. The most common form is random row-level splitting of network-flow records: because flows from the same session, host pair, or adjacent time window are highly correlated, random splits place nearly identical records into both training and test partitions. This allows models to achieve near-perfect scores by recognising identity patterns—specific IP addresses, port combinations, or session fingerprints—present in both sets, rather than learning generalisable attack behaviours [21].

A second form of leakage arises from identifier features: including raw IP addresses, port numbers, URIs, certificate subjects, and DNS query strings allows models to memorise which specific sources and destinations were associated with attacks in the training data. Such models fail when deployed against new infrastructure. Yet

many published studies include these features without discussion or ablation [18]. HYDRA addresses both issues through deployment-realistic temporal splits and explicit feature regime comparison, building directly on the evidence that excluding IP/port identifiers is necessary for deployment-realistic evaluation [18, 41].

2.8.2 Alert Fatigue and False Positive Management

Unsupervised and anomaly-based detection methods, despite their advantages for zero-day detection, tend to generate high false positive rates in dynamic operational environments where legitimate but rare behaviours are flagged as anomalous. Alert fatigue is consistently identified as a major barrier to effective SOC operation: when analysts face hundreds of daily false alerts, they develop alert blindness, increasing the probability that genuine threats are dismissed alongside benign activity [13].

2.8.3 Binary-Only Detection and Limited Attack Specificity

The majority of published IDS models are evaluated solely on binary classification—attack versus normal—despite the fact that operational response differs substantially depending on whether the detected threat is a denial-of-service attack, ransomware, lateral movement, or data exfiltration. Systems that raise an undifferentiated “attack” alert provide limited decision support; analysts must perform additional investigation to determine the threat type before prioritising response. HYDRA’s dual-task design reflects this operational requirement directly.

2.8.4 Explainability as a Qualitative Afterthought

Even among IDS studies that incorporate explainability methods, explanations are almost universally presented as static global feature importance plots or single illustrative SHAP waterfall charts, without any systematic evaluation of stability, fidelity, or coverage. This approach cannot determine whether the explanations are reliable enough for analyst use. Without systematic measurement of these properties, explainability in IDS remains performative rather than functional. HYDRA is designed to address this gap directly.

2.8.5 Adversarial Robustness and Concept Drift

Security ML models operate in an adversarial environment in which sophisticated attackers may actively attempt to evade detection or degrade model performance through data poisoning of training pipelines [27, 59]. In addition, concept drift—the statistical evolution of network behaviour due to new software, changing user patterns, or infrastructure changes—can render previously accurate models obsolete

without retraining [23]. Existing IDS prototypes rarely incorporate drift detection or adversarial hardening as integrated components. HYDRA’s modular architecture supports periodic re-evaluation and is designed to be extended with drift detection in future work.

2.9 Technology Survey

2.9.1 Streaming Infrastructure: Apache Kafka

Apache Kafka is an open-source distributed event streaming platform widely used in production data infrastructure for high-throughput, fault-tolerant log ingestion [31]. Kafka’s partitioned log architecture supports horizontal scalability and its append-only storage model supports both real-time streaming consumption and replay of historical data, enabling offline model retraining on the same event stream that feeds production inference. For cybersecurity applications, Kafka serves as the central event bus that decouples log producers from analytic consumers, enabling multiple detection models to consume the same stream independently and in parallel [31].

A practical challenge for real-time IDS is the latency–throughput trade-off inherent in Kafka’s batching model. Empirical evaluation of Kafka in distributed systems contexts demonstrates that end-to-end latency below two seconds is achievable at throughput levels of tens of thousands of records per second with appropriate batch-size and consumer configuration [5, 16]. Escribano-Barbero et al. [16] further propose the Kafka-ML framework, which manages the full lifecycle of ML models trained on and deployed against Kafka streams, reducing the serialisation overhead of integrating batch-oriented training frameworks with streaming inference.

2.9.2 Machine Learning Frameworks

Scikit-learn [47] provides mature, well-documented implementations of Random Forest, Gradient Boosted Trees, Logistic Regression, Isolation Forest, and the preprocessing components used in HYDRA’s tabular pipeline. PyTorch [46] provides the deep learning infrastructure used for the GNN model, with PyTorch Geometric [20] supplying graph learning utilities. PyTorch is selected over TensorFlow for this project based on its flexibility for custom architecture experimentation and its more Pythonic API.

2.9.3 Explainability Libraries

TreeSHAP is available through the SHAP Python library [51], which provides consistent implementations for tree-based models trained in scikit-learn, XGBoost, Light-

GBM, and CatBoost. GNNExplainer is available through PyTorch Geometric, which also provides Integrated Gradients attribution through its Captum integration layer.

2.9.4 Visualisation and Dashboard Tools

Two complementary tools are standard in the AIOps dashboard landscape. Grafana is an open-source observability platform for real-time time-series monitoring, integrating with databases such as InfluxDB to display live metrics, anomaly scores, and alert rates in configurable dashboards [49]. Streamlit is an open-source Python framework for building interactive data applications; its tight integration with standard Python data science libraries—pandas, Plotly, NetworkX, and Pyvis—makes it well-suited to the analyst workbench component of HYDRA, where interactive exploration of individual alerts, explanation records, and associated graph visualisations is needed [48]. Graph rendering and Neo4j integration allow analysts to explore attack graphs and causal paths visually [44].

2.10 Data Survey

2.10.1 Overview

HYDRA’s evaluation requires datasets that collectively exercise both its binary detection and multi-class classification tasks, its explainability evaluation protocols, and its graph-based detection component. Three datasets are used: TON_IoT as the primary benchmark, CIC-IoT2023 as a secondary generalisation dataset, and LogHub as a supporting reference for log-based analysis. Table 2.3 summarises their key characteristics.

Table 2.3: Summary of datasets relevant to HYDRA.

Dataset	Source	Scale	Attack Types	Primary Use
TON_IoT	Moustafa [42]	Millions of flows	9 types	Primary benchmark
CIC-IoT2023	CIC [9]	Millions of flows	33 types	Generalisation
LogHub	Zhu et al. [61]	Millions of entries	Failures, anomalies	Log reference
LANL	Turcotte et al. [57]	Billions of events	Insider, lateral	Context

2.10.2 TON_IoT (Primary Dataset)

The TON_IoT (Telemetry, Operating Systems, and Networking for IoT/IIoT) dataset collection, published by Moustafa [42] at the University of New South Wales Canberra, is among the most widely cited modern benchmarks for IoT intrusion detection. The dataset was collected from a realistic testbed environment comprising IoT sensors,

Linux and Windows hosts, and network monitoring infrastructure, and combines sensor telemetry, OS logs, and network traffic captured through Zeek-style flow monitoring.

The network-flow component, which is the primary modality used in HYDRA, contains millions of labelled records spanning nine attack categories: Scanning, DoS, DDoS, Injection, Backdoor, XSS, Ransomware, Password, and Man-in-the-Middle. Each record contains both a binary label (attack or normal) and a categorical attack-type label, supporting HYDRA’s dual-task design directly.

Known challenges with TON_IoT include significant class imbalance across attack types, potential artefacts from synthetic attack injection, and a risk of label leakage through identifier features that encode lab topology. An explainable universal feature set study [18] demonstrates that excluding IP-based identifiers substantially reduces reported accuracy but produces more deployment-realistic results. Analysis of ToN-IoT [41] confirms that deep learning models evaluated with appropriate temporal splits achieve lower but more credible performance than those evaluated with random row splits.

2.10.3 CIC-IoT2023 (Secondary Generalisation Dataset)

CIC-IoT2023, published by the Canadian Institute for Cybersecurity [9], is a large and recent IoT benchmark containing traffic from 33 distinct attack categories, including a broad range of DDoS variants, web attacks, brute-force techniques, and network scanning. Its greater attack diversity and more recent collection date compared to TON_IoT make it a useful generalisation testbed: models and explanation pipelines trained on TON_IoT are evaluated against CIC-IoT2023 to assess how well both detection performance and explanation quality transfer to a different dataset with overlapping but not identical attack types.

A practical challenge in using CIC-IoT2023 for generalisation testing is feature mapping: the two datasets use partially overlapping but not identical feature sets, requiring careful alignment of common behavioural features. HYDRA’s behaviour-only feature regime simplifies this alignment by restricting evaluation to features that are present and consistently defined in both datasets.

2.10.4 LogHub (Supporting Reference)

The LogHub collection [61] aggregates labelled log datasets from a variety of real systems including HDFS, BlueGene/L (BGL), Thunderbird, and several others. These datasets are standard benchmarks for log anomaly detection methods, including DeepLog [15], LogBERT [25], LogFiT [37], and ALogSCAN [1]. Within HYDRA,

LogHub serves as a reference point for evaluating the semantic processing pipeline rather than as a primary source for intrusion detection evaluation.

Limitations include heuristic labelling in some cases, low semantic diversity in certain log sources, and the age of some datasets which predate modern cloud-native architectures. These limitations mean that LogHub provides a controlled benchmarking environment but is not a sufficient surrogate for production network security log analysis.

2.10.5 LANL Unified Host and Network Dataset (Contextual Reference)

The LANL Unified Host and Network Dataset [57] contains tens of billions of de-identified authentication, process, and network events collected over 58+ days from a real enterprise network at Los Alamos National Laboratory, with embedded Red Team (simulated attacker) activities. It is widely regarded as the gold standard for insider threat and lateral movement research.

Within HYDRA, the LANL dataset serves a contextual rather than primary role: its characteristics inform the design of the graph construction approach and the lateral movement prediction methodology, and it provides important precedent for temporal GNN-based detection [34, 36].

2.10.6 Data Preparation Principles

Across all datasets, HYDRA applies a consistent data engineering pipeline. Log parsing uses template extraction tools such as Drain and Spell to convert raw log messages into structured event type sequences. Feature extraction derives numerical features for flows and aggregated metrics, and computes presence flags for sparse protocol-specific fields. Class imbalance is addressed through stratified sampling, SMOTE-based oversampling, and Focal Loss during training, following best practice from FIGS [21] and GAN-based synthetic data augmentation [24]. Temporal splitting ensures that training and test partitions respect time-ordering, avoiding unrealistically optimistic performance due to data leakage across time boundaries. All preprocessing transformations are fitted exclusively on training data and applied without modification to validation and test partitions.

2.11 Summary

The literature establishes four substantive conclusions that collectively motivate HYDRA’s design. First, ensemble tree methods—Random Forest, XGBoost, and

Gradient Boosted Trees—dominate tabular IDS benchmarks, yet reported results are frequently inflated by identifier-based feature leakage and random row-level splitting. Without controlling for these confounds, claimed performance gains conflate genuine detection capability with memorisation of lab topology, making deployment performance unpredictable. This finding directly motivates HYDRA’s dual-regime evaluation design, in which behaviour-only and identifier-inclusive feature sets are evaluated under temporal splits.

Second, SHAP TreeExplainer provides more stable and theoretically grounded attributions than LIME for tree models on tabular data, as its exact Shapley computation avoids the sampling variance that makes LIME explanations non-deterministic. However, LIME’s model-agnostic nature makes it the only viable cross-reference explanation method across heterogeneous model families—a property TreeExplainer cannot satisfy. This tension motivates HYDRA’s four-method XAI portfolio.

Third, no existing study applies a unified, quantitative XAI evaluation protocol across multiple model families on both binary and multi-class IoT intrusion detection under leakage-controlled conditions, nor measures whether explanation quality generalises across datasets. This gap is the primary research contribution HYDRA addresses. Chapter 3 describes the methodology through which these conclusions are operationalised into a reproducible experimental framework.

Chapter 3

Discussion

3.1 Recap of Key Results

[To be completed.]

3.2 Interpretation of Results

[To be completed.]

3.3 Practical Implications

[To be completed.]

3.4 Reflection on Research Questions

[To be completed.]

3.5 Limitations

Several limitations constrain the scope and generalisability of these findings. The first concerns the XAI portfolio. The CNN-LSTM model was designed and specified in Chapter 3 but not fully evaluated within the project timeframe; consequently, Integrated Gradients attributions for deep learning architectures could not be assessed, and the five-criterion protocol was applied exclusively to tabular models. Conclusions about explanation stability and fidelity therefore cannot be extended to neural architectures without further experimentation.

The second structural limitation is the scoping of the Graph Neural Network to future work. GNNExplainer was not implemented, and graph-level explanation quality—including subgraph stability and edge-feature fidelity—was not assessed.

The contribution of relational structure to both detection performance and explanation plausibility remains an open empirical question.

A third limitation concerns dataset provenance. Both TON_IoT and CIC-IoT2023 are laboratory-captured datasets that may not reflect the noise characteristics, class drift, and infrastructure diversity of live SOC deployments. Reported performance figures should therefore be interpreted as upper bounds on expected operational performance.

Fourth, the OXS composite score assigns equal weights of 0.2 to each criterion. This weighting is heuristic; rigorous calibration would require a structured analyst survey to elicit operational priority judgements, which is identified as future work in Section 4.3.

Fifth, LIME stability was measured on 500 randomly sampled test instances due to computational constraints. For rare attack classes this sample may be insufficient to produce stable estimates, and larger-scale measurement could yield materially different results.

Finally, SMOTE oversampling was applied to the training set to address class imbalance. Fidelity metrics on rare classes may therefore partially reflect the structure of synthetic instances rather than genuine traffic patterns, introducing a degree of optimism into plausibility scores for minority categories.

Chapter 4

Conclusions

4.1 Summary of Contributions

This dissertation made three contributions. First, a reproducible dual-task intrusion detection pipeline was implemented and evaluated across five tabular models on TON_IoT and CIC-IoT2023 under two leakage-controlled feature regimes, achieving a binary PR-AUC of [PLACEHOLDER] and a multi-class macro-F1 of [PLACEHOLDER] under the behaviour-only regime. Second, a unified five-criterion XAI evaluation protocol—covering faithfulness, stability, simplicity, plausibility, and timeliness—was applied systematically across all models, yielding an OXS-ranked comparison in which [PLACEHOLDER] achieved the highest overall explanation quality score of [PLACEHOLDER]. Third, the first empirical measurement of the interaction between identifier-based leakage and explanation quality demonstrated that including identifier features inflated RMA@5 by [PLACEHOLDER] percentage points while reducing Spearman stability by [PLACEHOLDER].

4.2 Achievement of Aim

The overall aim of this project was substantially achieved. HYDRA was designed, implemented, and evaluated as a dual-task intrusion detection framework, producing per-alert explanations validated against five quantitative criteria on both TON_IoT and CIC-IoT2023. Binary and multi-class detection were addressed simultaneously, and the leakage-controlled evaluation design ensured that reported results of [PLACEHOLDER] PR-AUC and [PLACEHOLDER] macro-F1 reflect deployment-realistic conditions. One area of partial achievement is noted: the CNN-LSTM component was designed and fully specified but not evaluated within the project timeframe, leaving the deep learning branch of the XAI portfolio incomplete, as discussed in Section 3.5.

4.3 Future Work

[To be completed.]

4.4 Closing Remarks

[To be completed.]

Bibliography

- [1] Liu, Y., et al. (2025) ALogSCAN: A Self-Supervised Dual Network for Adaptive and Timely Log Anomaly Detection in Clouds. *IEEE Transactions on Network and Service Management*, 22(1), pp. 1105–1119. doi:10.1109/TNSM.2024.3511554.
- [2] Alsaedi, A., et al. (2020) TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems. *IEEE Access*, 8, pp. 165130–165150.
- [3] Alvarez-Melis, D. and Jaakkola, T.S. (2018) On the Robustness of Interpretability Methods. *ICML Workshop on Human Interpretability in Machine Learning*. Available at: <https://arxiv.org/abs/1806.08049> [Accessed 5 January 2026].
- [4] Elsayed, M.S., et al. (2025) Forecasting Future DDoS Attacks Using Long Short-Term Memory Networks. *arXiv preprint arXiv:2509.02076*. Available at: <https://arxiv.org/abs/2509.02076> [Accessed 2 December 2025].
- [5] Wang, G., et al. (2021) Apache Kafka in the Context of Distributed Systems Architectures: A Systematic Evaluation. *IEEE Transactions on Network and Service Management*, 18(4), pp. 4628–4641.
- [6] Bauer, L., Garriss, S. and Reiter, M.K. (2017) Policy Graphs: A Tractable Representation for Complex Security Policies. *ACM Conference on Computer and Communications Security (CCS)*.
- [7] Bhatt, U., et al. (2020) Evaluating and Aggregating Feature-based Model Explanations. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*.
- [8] Breiman, L. (2001) Random Forests. *Machine Learning*, 45(1), pp. 5–32.
- [9] Canadian Institute for Cybersecurity (2023) CIC-IoT2023 Dataset. Available at: <https://www.unb.ca/cic/datasets/> [Accessed 2 December 2025].
- [10] Chen, T. and Guestrin, C. (2016) XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- [11] Datategy (2025) Why AIOps Is Key to Cyber Threat Detection in Defence

[industry report]. Available at: <https://www.datategy.net/2025/07/02/why-aiops-is-key-to-cyber-threat-detection-in-defense/> [Accessed 2 December 2025].

- [12] DeYoung, J., et al. (2020) ERASER: A Benchmark to Evaluate Rationalized NLP Models. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pp. 4443–4458.
- [13] Aldweesh, A., Derhab, A. and Emam, A.Z. (2020) Deep Learning Approaches for Anomaly-based Intrusion Detection Systems: A Survey, Taxonomy, and Prospective Outlook. *Computers & Security*, 99, p. 101996.
- [14] Ahmad, Z., et al. (2021) Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches. *Transactions on Emerging Telecommunications Technologies*, 32(1), e4150.
- [15] Du, M., et al. (2017) DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, pp. 1285–1298.
- [16] Escribano-Barbero, F., et al. (2023) The Orchestration of Machine Learning Frameworks with Data Streams and GPU Acceleration in Kafka-ML. *Future Generation Computer Systems*, 145, pp. 1–14.
- [17] Adadi, A. and Berrada, M. (2018) Peeking Inside the Black Box: A Survey on Explainable Artificial Intelligence (XAI). *IEEE Access*, 6, pp. 52138–52160. doi:10.1109/ACCESS.2018.2870052.
- [18] Lanvin, M., Giura, P., Mallouli, W. and de Oca, E.M. (2023) Errors in the CI-CIDS2017 Dataset and the Significant Differences in Detection Performances it Makes. *International Conference on Risks and Security of Internet and Systems (CRiSIS)*.
- [19] Fang, P., et al. (2022) Back-Propagating System Dependency Impact for Attack Investigation. *31st USENIX Security Symposium*. Available at: <https://www.usenix.org/system/files/sec22-fang.pdf> [Accessed 11 December 2025].
- [20] Fey, M. and Lenssen, J.E. (2019) Fast Graph Representation Learning with PyTorch Geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [21] Lanvin, M., et al. (2023) FIGS: A Realistic Intrusion-Detection Framework for Highly Imbalanced IoT Environments. *Electronics*, 14(14), article 2917. Avail-

able at: <https://www.mdpi.com/2079-9292/14/14/2917> [Accessed 11 December 2025].

- [22] Fortinet (2025) 2025 Global Threat Landscape Report [industry report]. Available at: <https://www.fortinet.com/resources/cyberglossary/artificial-intelligence-in-cybersecurity> [Accessed 15 November 2025].
- [23] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. and Bouchachia, A. (2014) A Survey on Concept Drift Adaptation. *ACM Computing Surveys*, 46(4), article 44.
- [24] Alabsi, B.A., et al. (2023) CNN-CNN: Dual Convolutional Neural Network Approach for Feature Selection and Attack Detection on Internet of Things Networks. *Sensors*, 23(14), article 6507.
- [25] Guo, H., Yuan, S. and Wu, X. (2021) LogBERT: Log Anomaly Detection via BERT. *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, pp. 1–8.
- [26] Popoola, S.I., et al. (2024) Federated Deep Learning for Zero-Day Botnet Attack Detection in IoT-Edge Devices. *IEEE Transactions on Industrial Informatics*, 18(6), pp. 3988–3997.
- [27] Goldblum, M., et al. (2022) Dataset Security for Machine Learning: Data Poisoning, Backdoor Attacks, and Defenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(2), pp. 1563–1580.
- [28] Malik, J., et al. (2024) Hybrid Machine Learning Algorithm for Intrusion Detection Systems. *IEEE Access*, 12, pp. 74391–74407. doi:10.1109/ACCESS.2024.3406053.
- [29] Talukdar, V., et al. (2024) Advanced Intrusion Detection System Through Hybrid Integration of Random Forest and Deep Learning with Artificial Neural Networks. *IEEE Access*, 12, pp. 182661–182675. doi:10.1109/ACCESS.2024.3512553.
- [30] Imrana, Y., et al. (2022) A Bidirectional LSTM Deep Learning Approach for Intrusion Detection. *Expert Systems with Applications*, 185, p. 115524.
- [31] Apache Software Foundation (2024) Apache Kafka Documentation. Available at: <https://kafka.apache.org/> [Accessed 2 December 2025].
- [32] Kanerika (2025) AIOps Platforms for Cybersecurity in 2025: Trends and Key Capabilities [industry report]. Available at: <https://kanerika.com/blogs/aioops-platforms-for-cybersecurity/> [Accessed 2 December 2025].
- [33] Ke, G., et al. (2017) LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.

- [34] King, I., et al. (2023) Euler: Detecting Network Lateral Movements via Scalable Temporal Link Prediction. *Network and Distributed Systems Security (NDSS) Symposium 2023*.
- [35] Liu, F.T., Ting, K.M. and Zhou, Z.H. (2008) Isolation Forest. *2008 Eighth IEEE International Conference on Data Mining*. IEEE, pp. 413–422.
- [36] Lo, W.W., et al. (2023) Hermes: Lateral Movement Detection via Heterogeneous Graph Attention Networks. *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*.
- [37] Qi, J., et al. (2023) LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models. *IEEE Transactions on Knowledge and Data Engineering*, 36(5), pp. 2290–2302.
- [38] Malhotra, P., et al. (2016) LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. *Proceedings of the ICML 2016 Anomaly Detection Workshop*.
- [39] Lundberg, S.M. and Lee, S.-I. (2017) A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.
- [40] Lundberg, S.M., et al. (2020) From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*, 2(1), pp. 56–67.
- [41] Ferrag, M.A., et al. (2022) Analysis of ToN-IoT, UNW-NB15, and Edge-IIoT Datasets Using Deep Learning in Cybersecurity for IoT. *Applied Sciences*, 12(19), article 9572. doi:10.3390/app12199572.
- [42] Moustafa, N. (2021) A New Distributed Architecture for Evaluating AI-based Security Systems at the Edge: Network TON_IoT Datasets. *Sustainable Cities and Society*, 72, p. 102994.
- [43] Munkhdorj, T., et al. (2017) Cyber Attack Prediction Using Social Data Analysis. *2017 IEEE 7th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE.
- [44] Robinson, I., Webber, J. and Eifrem, E. (2015) *Graph Databases: New Opportunities for Connected Data*. 2nd ed. O'Reilly Media.
- [45] Park, S., Kim, S. and Lee, J. (2023) Anomaly Detection in Industrial Control Systems Using LSTM-Autoencoder. *IEEE Access*, 11, pp. 55803–55812.
- [46] Paszke, A., et al. (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.
- [47] Pedregosa, F., et al. (2011) Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.

- [48] Streamlit Inc. (2023) *Streamlit: A Faster Way to Build and Share Data Apps* [Software, v1.28]. Available at: <https://streamlit.io> [Accessed 20 October 2025].
- [49] Grafana Labs (2023) *Grafana: The Open Observability Platform* [Software, v10.0]. Available at: <https://grafana.com> [Accessed 18 October 2025].
- [50] Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144.
- [51] Lundberg, S.M. (2022) *SHAP: A Unified Approach to Explaining the Output of any Machine Learning Model*. GitHub/PyPI library, v0.42. Available at: <https://github.com/shap/shap> [Accessed 2 December 2025].
- [52] Sundararajan, M., Taly, A. and Yan, Q. (2017) Axiomatic Attribution for Deep Networks. *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, pp. 3319–3328.
- [53] Sönmez, Y. and Özdemir, S. (2023) Predictive Intrusion Detection: A Survey of Methods, Metrics, and Datasets. *IEEE Access*, 11, pp. 86254–86278. doi:10.1109/ACCESS.2023.3304008.
- [54] Le, V.H. and Zhang, H. (2023) Evaluation of Deep Learning Techniques for Log-based Anomaly Detection. *ACM Transactions on Software Engineering and Methodology*, 33(1), article 8.
- [55] Tan, J. and Anagnostopoulos, C. (2024) Unified Parallel Semantic Log Parsing Based on Causal Graph Construction for Attack Attribution. *Expert Systems with Applications*, 238, p. 121876.
- [56] Dang, D., et al. (2023) AIOps: Real-World Challenges and Research Innovations. *IEEE Software*, 40(3), pp. 45–52.
- [57] Turcotte, M.J.M., Kent, A.D. and Hash, C. (2017) Unified Host and Network Data Set. In: *Data Analytics and Decision Support for Cybersecurity*. Springer, pp. 1–22.
- [58] Nassif, A.B., et al. (2021) Machine Learning for Anomaly Detection: A Systematic Review. *IEEE Access*, 9, pp. 78658–78700.
- [59] Schwarzschild, A., et al. (2021) Just How Toxic is Data Poisoning? A Unified Benchmark for Backdoor and Data Poisoning Attacks. *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, pp. 9382–9392.
- [60] Ying, Z., et al. (2019) GNNExplainer: Generating Explanations for Graph Neural Networks. *Advances in Neural Information Processing Systems*, 32.

- [61] Zhu, J., et al. (2023) Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. *IEEE International Symposium on Software Reliability Engineering (ISSRE)*. IEEE.
- [62] Islam, R., et al. (2024) E-XAI: Evaluating Black-Box Explainable AI Frameworks for Network Intrusion Detection. *IEEE Access*, 12, pp. 19859–19884.
- [63] Srivastava, G., et al. (2024) An Explainable AI Framework for Intrusion Detection Systems. *Applied Sciences*, 14(2), article 753. doi:10.3390/app14020753.
- [64] Bhatt, P., et al. (2025) Explainable AI-Driven Intrusion Detection in IoT: A Comparative Analysis of SHAP and LIME. *Frontiers in Computer Science*, 7, article 1509631. doi:10.3389/fcomp.2025.1509631.
- [65] Viirlaid, A., et al. (2025) Evaluation of Explainable AI Approaches for Network Intrusion Detection Systems in Security Operations Contexts. *Baltic Journal of Modern Computing*, 13(2), pp. 183–201.

Chapter A

Feature Groups for TON_IoT

The following table summarises the feature classification used in HYDRA’s two feature regimes.

Table A.1: TON_IoT feature groups and inclusion in the two evaluation regimes.

Feature Group	Features	Regime A (Behaviour)
Behavioural	<code>duration</code> , <code>src_bytes</code> , <code>dst_bytes</code> , <code>src_pkts</code> , <code>dst_pkts</code> , <code>missed_bytes</code> <code>src_ip_bytes</code> , <code>dst_ip_bytes</code>	Included
Metadata	<code>proto</code> , <code>service</code> , <code>conn_state</code>	Included
Sparse protocol flags	DNS/HTTP/SSL presence indicators	Included
Identifiers (leakage risk)	<code>src_ip</code> , <code>dst_ip</code> , <code>ports</code> , <code>URIs</code> , <code>cert subjects/issuers</code> , <code>user agents</code>	Excluded

Chapter B

Explainability Metric Formulae

Faithfulness (Comprehensiveness). For each alert x with top- k attributed feature set $T(x)$, create a perturbed version $\tilde{x}$ by replacing features in $T(x)$ with class-conditional median values. Comprehensiveness is:

$$C(x) = p(y=1 \mid x) - p(y=1 \mid \tilde{x})$$

Reported as mean C across test alerts, compared against a random- k perturbation control.

Stability (two separate protocols). Stability addresses two distinct questions that are operationalised as separate experiments. Alvarez-Melis and Jaakkola [3] demonstrate that explanation methods can be sensitive both to small input variations and to differences in model parameters; accordingly, both sources of instability must be measured independently.

Protocol S1 — Seed-retraining stability (addresses RQ3). Each model is trained $S = 5$ times with different random seeds on the same training data, holding the test set and alert selection fixed. For each seed i , extract the top- k feature importance ranking F_i for every alert in a fixed evaluation set A . Spearman rank correlation $\rho(F_i, F_j)$ is computed for all pairs (i, j) :

$$\rho(F_i, F_j) = 1 - \frac{6 \sum_{\ell} d_{\ell}^2}{k(k^2 - 1)}$$

where d_{ℓ} is the rank difference for feature ℓ between runs i and j . Report mean $\pm$ standard deviation across all $\binom{S}{2} = 10$ pairs and all alerts. High ρ indicates that explanation outputs are consistent regardless of which particular local optimum the model converges to.

Protocol S2 — Noise-perturbation robustness (addresses RQ4 robustness sub-question). Using a single fixed trained model, add small Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ to continuous input features (with $\sigma = 0.01 \times \text{feature std}$) and re-extract the top- k ranking

for each alert. Spearman correlation $\rho(F_{\text{orig}}, F_{\text{noisy}})$ is computed between the original and perturbed rankings. Report mean $\pm$ standard deviation across $R = 20$ noise draws and all alerts. High ρ indicates that small input perturbations do not reorder the explanation features substantially, reflecting the smoothness of the attribution surface.

These two protocols are complementary: S1 measures robustness to training stochasticity; S2 measures robustness to input noise. Results are reported separately in the evaluation.

Simplicity. k_{90} is the minimum number of top-attributed features needed to account for 90% of total attribution mass, normalised by total feature count n :

$$k_{90} = \min \left\{ k : \frac{\sum_{i=1}^k a_{(n-i+1)}}{\sum_{i=1}^n a_{(i)}} \geq 0.90 \right\} / n$$

where $a_{(i)}$ are attribution magnitudes sorted in ascending order. *Note on comparability across regimes:* because Regime A (behaviour-only) and Regime B (identifier-inclusive) have different feature counts n , the normalised k_{90} fractions are comparable in direction but not in absolute magnitude. Results are therefore reported per-regime with explicit feature counts stated.

The Gini coefficient G of the attribution distribution is:

$$G = \frac{2 \sum_{i=1}^n i \cdot a_{(i)}}{n \sum_{i=1}^n a_{(i)}} - \frac{n+1}{n}$$

where $a_{(i)}$ are attribution magnitudes sorted in ascending order.

Plausibility. $\text{RMA}@k$ (Relevant-feature Match Accuracy at k) is:

$$\text{RMA}@k = \frac{|\text{top-}k \text{ features} \cap \mathcal{R}_c|}{k}$$

where $\mathcal{R}_c$ is the reference set for attack class c . Reference sets are defined in advance from domain knowledge based on the IDS and network security literature, and are fixed before any model is trained or evaluated. Table B.1 lists the reference features used for each of the nine TON_IoT attack categories.

Timeliness. Wall-clock time in milliseconds to generate a complete explanation record for a single alert, measured over 500 randomly sampled test alerts and reported as mean $\pm$ standard deviation. Measurements are made on a single CPU core to provide a reproducible baseline independent of GPU availability, as explanation generation in SOC deployments may run on commodity hardware. Methods are compared within the same hardware environment.

Table B.1: Expert-defined reference feature sets $\mathcal{R}_c$ for RMA@ k evaluation, one per TON_IoT attack category. Features are drawn from standard network security domain knowledge [2, 18, 41].

Attack type	Reference features $\mathcal{R}_c$
DoS / DDoS	src_bytes, dst_bytes, src_pkts, dst_pkts, duration, conn_state
Scanning	dst_pkts, duration, conn_state, service, proto
Ransomware	src_bytes, dst_bytes, duration, missed_bytes, conn_state
Backdoor	duration, src_bytes, dst_bytes, conn_state, proto
Injection	src_bytes, dst_bytes, service, conn_state, proto
XSS	src_bytes, dst_bytes, service, conn_state, duration
Password	duration, src_pkts, dst_pkts, conn_state, proto
Man-in-the-Middle	src_bytes, dst_bytes, src_pkts, dst_pkts, conn_state
Normal	N/A (not applicable for attack-type plausibility)

Chapter C

Acknowledgement of Generative AI Use

This dissertation was prepared by the author. Generative AI tools (OpenAI ChatGPT) were used for text editing and structural suggestions during drafting. All technical ideas, design decisions, experimental methodology, analysis, and written conclusions are the author's own work, produced in accordance with the University of Bath's Type B guidelines on AI-assisted assessment (CM32017 Individual Project Handbook, 2025).